Verification and Validation Report: SFWRENG 4G06 - Capstone Design Project

Team #7, Wardens of the Wild
Felix Hurst
Marcos Hernandez-Rivero
BoWen Liu
Andy Liang

April 6, 2026

Revision History

Date	Version	Notes
March 4, 2026	1.0	Initial version

Contents

This document will report on the results of the Verification & Validation (VnV) Plan. After covering the results, we will discuss the changes we intend to make to improve the final version of the project.

1 General Information

The VnV Plan covers plans for evaluating key documents and artifacts of our project. This includes the Software Requirements Specification (SRS), the Module Guide (MG) which served as our design document, the VnV Plan itself, the implementation of the design, and the system as a whole. The plan also included a list of system tests and unit tests to be performed on the project and its source code.

For a full description of the plan, refer to the original **VnV Plan**.

2 Evaluation of Documentation and Artifacts

This section will cover the evaluation of our key design-related documentation, as well as informal demonstration and feedback on our implementation.

2.1 Overview

A large portion of feedback received, especially from classmates designated as peer reviewers, covered formatting issues in our documentation that caused difficulties with readability and clarity. However, we also received plenty of useful feedback for the content and substance of our documents and artifacts. Overall, our project had decent verification but weak validation, requiring us to make significant changes to our plans of what we are creating with this project.

2.2 SRS Verification

This refers to the **SRS**.

Feedback gathered resulted in many fixes being made to the SRS, but these were primarily formatting fixes. The content was deemed well-suited to the project, covering key aspects of the project appropriately, and so we moved forward without making major changes to the requirements.

2.3 Design Verification

This refers to the **MG**.

Feedback gathered included some significant points of low clarity in the design documentation, namely the lack of modules covering the slime mold algorithm, a crucial aspect of our project. We swiftly amended this issue, adding new, clear modules for the slime mold algorithm and slime mold entities our project will use. The remaining feedback received majorly covered formatting and typographical errors, and indicated that the content of the other modules listed was appropriate. Specifications were easy to understand and usable by other external teams, as evidenced by the implementation of our Pixelation Module (M13) by Team 11.

2.4 Verification and Validation Plan Verification

This refers to the **VnV Plan**.

Feedback gathered indicated that some of our sections lacked clarity, on top of having poor formatting. These issues were amended, restoring a clear and effective plan that properly covers all project documents and artifacts.

2.5 Implementation Verification

Detailed descriptions of test results for system and unit tests are written in Sections 3 and 4 respectively.

Static analysis showed that for the majority of scripts, code quality was to an acceptable degree, with minimal necessity to refactor for optimization purposes. However, one particular set of scripts responsible for water simulation was a cause for major performance issues. It was found through static analysis that these scripts were running background event checks far too many times per second, slowing down the system. This issue was quickly corrected.

2.6 Automated Testing and Verification Tools

Our team implemented Unity C# Linter to check for defects in our source code, and to ensure a certain level of code quality. This tool was used throughout development, and helped us identify and fix issues in our code as they arose. Furthermore, tex files have compile checkers that were used to

ensure the formatting and structure of our documents were correct, and to catch any errors in the tex code.

2.7 Software Validation

A large volume of useful and thought-provoking feedback was given to us from the PoC and Rev 0 demonstrations we presented. Through speaking with some of our stakeholders, including our instructor, TA and supervisors, we were encouraged to focus our game direction and to align our efforts more closely to the goals that we had laid out for it in our prior documents, such as the **Problem Statement and Goals** document.

This led to major changes in our development plans. As a team, we held meetings to discuss the new direction we wanted to take our project, which involved numerous votes and compromises to ensure every team member was on board with the plan. This included a renewed focus on the element of eliciting emotions from our users, as we present this project which is both artistic and educational, showcasing the unique properties of the slime mold while simultaneously teaching about cooperation with and protection of nature.

We have also been conducting a Usability Survey, to directly connect with our end-users, the players of our game. The results of this survey can be found in the **Usability Report** document.

3 System Tests

3.1 Tests for Functional Requirements

The following tests verify that the core gameplay mechanics behave as intended, covering object destruction, slime mold interactions, and player movement.

3.1.1 ObjectDestructionTest-FRT01

Initial State: Player at beginning of level, with a destructible obstacle in their path. Player is 5 units away from the obstacle, and has the rifle tool selected.

Input: Select rifle tool, approach a destructible obstacle, aim, and fire the tool at the object.

Expected Output: A destructive ball is fired from the tool at the object. About one second after contact, the object breaks into smaller pieces. Note that distance does not impact destruction behaviour.

Actual Output: Same as expected.

3.1.2 SlimeMoldFunctionalityTest-FRT02

Initial State: Character at beginning of level, with a wood obstacle in their path, and a slime mold source near that obstacle.

Input: Select water ball tool, approach a wood obstacle, aim, and fire the tool at the object.

Expected Output: A water ball is fired from the tool at the object. After making contact with the wood obstacle, a nearby slime mold organism begins to traverse toward it. After a while, the slime mold decomposes the wood obstacle, destroying it entirely.

Actual Output: Same as expected.

3.1.3 SlimeSporeSpreadingTest-FRT03

Initial State: Character standing still, large number of spore particles spawned on the ground.

Input: Select spore fan tool, aim at the spore particles on the ground, and fire the tool at the spore particles.

Expected Output: Spore particles are accelerated away from the character, following a smooth, gentle path through the air, and landing on the ground a short distance away. Where spores land, they spawn new slime mold sources.

Actual Output: Mostly the same as expected, except that spore particles are accelerated too quickly.

3.1.4 SlimeWallClimbTest-FRT04

Initial State: Character at beginning of level, with a vertical surface (i.e. a wall) in their path, and a slime mold source near that wall.

Input: Select water ball tool, aim at the wall, and fire the tool at the wall. Wait a few seconds, then attempt to climb the wall.

Expected Output: The slime mold is attracted to the water on the wall. After covering enough of the wall, the wall becomes climbable, and the character ascends it successfully.

Actual Output: Untested. Functionality has yet to be implemented.

3.2 Tests for Non-Functional Requirements

The following tests evaluate the quality attributes of the system, including performance under load and the unpredictability of slime mold behaviour.

3.2.1 PerformanceTest-NFRT01

Initial State: Freshly loaded level.

Input: Repeated usage of tools in an attempt to spawn as many particles as possible and/or break obstacles into as many pieces as possible.

Expected Output: The frame rate will be carefully observed, and should stay within reasonable bounds, averaging 30 fps or higher.

Actual Output: Same as expected.

3.2.2 SlimeMoldRandomnessTest-NFRT02

Initial State: Freshly loaded level.

Input: 5 reloads of a single part of a level, each time aiming the water ball tool in the same place to direct the slime mold toward the first wood obstacle.

Expected Output: The slime mold should have subtly different, random behaviour each time. At marked debug intervals on the object, slime mold agents should be observed to take different paths, and to decompose the wood obstacle at different speeds, each time the test is run.

Actual Output: Same as expected.

3.3 Traceability Between Test Cases and Requirements

Refer to the original **VnV Plan**.

4 Unit Tests

The following unit tests isolate and validate individual components of the system, including physics, fluid simulation, and agent behaviour.

4.1 Tests

4.1.1 RaycastObjectSplitTest-UT01

Initial State: Rectangular object.

Input: Apply raycast through object, entering from one side and exiting out the other, with a straight line.

Expected Output: Remove chunk from object and generate it as a new separated object, with the shape determined by the placement of the raycast.

Actual Output: Same as expected.

4.1.2 WaterLimitTest-UT02

Initial State: An empty, enclosed area of terrain.

Input: A water spawner is placed inside this area.

Expected Output: Water falls from the spawner and pools up on the ground, but only to a certain capacity, after which it remains a steady volume of water directly above the ground.

Actual Output: Same as expected.

4.1.3 SlimeMoldAttractionTest-UT03

Initial State: Slime mold source in a relatively stationary state.

Input: Introduction of water particles near slime mold source.

Expected Output: Some slime mold agents begin to move toward the water source.

Actual Output: Same as expected.

4.2 Traceability Between Test Cases and Modules

Refer to the original **VnV Plan**.

5 Discussion of System and Unit Test Results

The vast majority of system and unit tests were successful. Our project is well-verified in terms of calculations and integrations between modules. The main point of possible improvement lies in the newest implemented features, such as the spore fan tool.

However, while our tests indicate success, much of our project still needs proper validation from our stakeholders. This will be most prominently showcased in our **Usability Report** document.

We currently believe the project is lacking substance and length. It is too small, and needs more for the player to experience, and a proper story to feel emotion from.

6 Changes Due to Testing

Parameters for certain modules such as the spore fan tool as well as the slime mold scripts still need adjustment for the best appearance that will reflect natural and realistic particle movement, which is important for the artistic and educational aspects of our project.

7 Code Coverage Metrics

The system and unit test suites together are designed to ensure near-100% **branch coverage** of the source code for all important functionality. We believe branch coverage is sufficient for our project given the time available, as it guarantees every major code block and control-flow path has been executed at least once, without the overhead of condition or multiple condition coverage.

The following breaks down how coverage is achieved across each major module:

- **Tool Usage**: FRT01, FRT02, and FRT03 collectively exercise all three tools (destructive ball, water ball, and spore fan), covering the branching logic for tool selection, projectile firing, and on-contact effects.
- **Destructible Object Behaviour**: FRT01 and UT01 together cover the two main destruction paths — gradual fracturing on impact and raycast-based splitting — ensuring both the fracture and object-generation branches are reached.
- **Slime Mold Behaviour**: FRT02, FRT03, NFRT02, and UT03 cover the key branches in slime mold logic, including agent attraction to water, traversal toward a target, decomposition of wood obstacles, and spore spreading and germination.

- **Fluid Simulation**: UT02 covers the branching logic for water accumulation, specifically the steady-state cap that prevents unbounded pooling, ensuring both the filling and capacity-reached branches are exercised.
- **Performance and Particle Systems**: NFRT01 exercises the high-load branches of the particle and physics systems by spawning the maximum feasible number of particles and fragments simultaneously.

Appendix — Reflection

The information in this section will be used to evaluate the team members on the graduate attribute of Reflection.

1. What went well while writing this deliverable?

2. What pain points did you experience during this deliverable, and how did you resolve them?

3. Which parts of this document stemmed from speaking to your client(s) or a proxy (e.g. your peers)? Which ones were not, and why?

4. In what ways was the Verification and Validation (VnV) Plan different from the activities that were actually conducted for VnV? If there were differences, what changes required the modification in the plan? Why did these changes occur? Would you be able to anticipate these changes in future projects? If there weren't any differences, how was your team able to clearly predict a feasible amount of effort and the right tasks needed to build the evidence that demonstrates the required quality? (It is expected that most teams will have had to deviate from their original VnV Plan.)

Team Response:

1. It was easy to draw conclusions about what our team had done right and what we had done wrong, as well as how to improve our project moving forward, using the feedback and recommendations we received.

2. It was difficult to collect information about some of our team's past meetings with each other, our instructor and TA, and our supervisors. Care needed to be taken to search through our Issue Tracker on GitHub as well as our chat history on Discord to find notes on past discussions we had that were part of the VnV process.

3. The most important aspect of our VnV Plan regarding speaking with our clients was our Usability Survey, an opportunity for alpha testers to play a demo of our game and submit their feedback on it. Formal demonstrations to our instructor and TA, and informal demonstrations to our supervisors, which were done for verification and validation of

documentation and artifacts, were also of high importance. In contrast, manual testing in the form of system and unit tests were done independently without client interaction, as these were technical verification processes and did not require client input.

4. Most of our original VnV Plan was modified to reflect more accurately feasible steps we could take to verify and validate our project. These modifications needed to be made because our team encountered setbacks in our development process, due to miscommunication, overestimation of what we could accomplish, and drama within the team. These setbacks meant that we had less time to dedicate to longer, more thorough VnV processes that may have involved more formal meetings with stakeholders and classmates, more sophisticated and lengthier test suites, automated testing, and more. Instead, we had to trim much of our initial plan down and focus on key VnV activities such as our key system and unit tests, and our Usability Survey. In future projects, we would likely be able to anticipate such changes sooner, having the experience to improve our estimation and time management, and the general knowledge of certain activities that require more time to invest in, or that are simply infeasible, such as how we dropped automated testing.